

Homework 4

Due: @11:55PM, Monday, December 9th

Name: _____Shikha Nukala_____ Uniqname: _____desigirl_____

1. Submit a pdf of your typed or handwritten homework on Gradescope.
2. Your answers should be neat, clearly marked, and concise. Typed work is recommended, but not required unless otherwise stated. Show all your work where requested, and state any special or non-obvious assumptions you make.
3. You may discuss your solution methods with other students, but the solutions you submit must be your own.
4. **Late Homework Policy:** Submissions turned in by 1:00 am the next day will be accepted but with a 5% penalty. Assignments turned in between 1:00 am and 11:55 pm will get a 30% penalty, and any submissions made after this time will not be accepted.
5. When submitting your answers to Gradescope you need to indicate what page(s) each problem is on to receive credit. The grader may choose not to grade the homework if answer locations are not indicated.
6. After each question (or in some cases question part), we've indicated which lecture number we expect to cover the relevant material. So “(L7)” indicates that we expect to cover the material in lecture 7 (but depending on your lecture, it may have been covered later)
7. **The last question is a group question.**
 - You may do it in a homework group of up to two students including yourself (yes, you can do them by yourself if you wish).
 - If you work in a group of two for these questions, list the name of the student you worked with in your assignment. Further, we suggest that you not split these problems up but rather work on the problem as a group.
 - Turn these group questions in as part of your individual submission.
 - **For these questions (and these questions ONLY) you are allowed to copy/paste solutions from the other student in your homework group.**
 - It is an honor code violation if a student is listed as contributing who did not actually participate in working on that problem.

Problem 1: Out of Control [16 points]

For the questions below, reference the following piece of LC2K assembly code: [\(L15\)](#)

```
lw    0    1    pos1
lw    0    2    neg1
lw    0    3    count
nor   1    1    4
loop  beq   0    3    fin        //beq0
      nor   3    3    5
      nor   4    5    5
      beq   0    5    even      //beq1
      add   6    1    6
      beq   0    0    next      //beq2
even  add   7    1    7
next  add   3    2    3
      beq   0    0    loop      //beq3
fin    halt
count .fill 5
pos1  .fill 1
neg1  .fill -1
```

1. Write the sequence of branch decisions for each beq instruction. Let “taken” be denoted as “T” and “not taken” as “N”. For example, a beq that is taken twice and then not taken once would have a sequence TTN. **[4]**

beq0	NNNNNT
beq1	NTNTN
beq2	TTT
beq3	TTTTT

How many extra cycles are added due to the following schemes by squashes, assuming the 5-stage pipeline from lecture? Show your work.

2. Speculate and Squash: Predict always taken [2]

If the prediction is correct and the branch is taken then no extra cycles are necessary in that scenario. If the prediction is incorrect and the branch is not taken then squash will occur in the execute stage and three cycles will be needed because there was wasted work done in the IF and ID and EX stages. Therefore the total number of cycles needed is $3 \times$ the number of incorrect predictions (the branch was not taken). In this case since there are 8 not taken instances we have $8 \times 3 = 24$ extra cycles.

3. Speculate and Squash: Predict backwards taken, forwards not taken [3]

Backwards branches are negative offsets and forward branches are positive offsets. If backwards is predicted as taken and the branch is taken then no extra cycles are needed. If forwards is predicted as not taken and the branch is not taken then no extra cycles are needed. If we have the other two scenarios then there is wasted work in three stages so for every mispredicted branch 3 extra cycles are needed. The first three branches are forwards and the last is backwards. If forwards is predicted not taken but in reality are taken 6 times, then the number of extra cycles is $6 \times 3 = 18$.

4. Dynamic branch predictor with separate 1-bit counter for each branch: Initialized to taken [3]

In this case since all branches are initialized to a taken counter then only the branches that are not taken will be mispredicted. For beq0 the first instance is mispredicted and then the counter flips so the next four are correctly predicted and the last is mispredicted. For beq1 there are only flips so it is wrongly predicted all five times. For beq2 and beq3 its always taken so always correctly predicted. Therefore there are 7 mispredictions so $3 \times 7 = 21$ extra cycles.

5. Dynamic branch predictor with separate 2-bit counter for each branch: Initialized to strongly taken [3]

Here every branch is initialized to strongly taken. Therefore the predictor will predict taken twice for each branch and after that if there are two consecutive mispredictions then the counter will change. For beq0 we have two consecutive mispredictions so the counter shifts on the third and is correct until the taken branch is hit then it has another misprediction. For beq1 it's incorrect for the not taken branches but correct for the taken branches. For beq2 and beq3 it is always correctly predicted. Therefore we have 6 mispredictions and $6 \times 3 = 18$ extra cycles.

Problem 2: A real page turner [20 points] (L22)

Consider a 42-bit, byte-addressable system that uses virtual memory. The system has a maximum of 128 GB of physical memory installed with a page size of 8 KB.

1. How many bits are used for the page offset? How many bits are used to index into the page table(s)? [4]

Page offset = $\log_2(8 \text{ KB}) = 13$, VPN bits = total bits - page offset = $42 - 13 = 29$

Page Offset: ____ 13 ____ Virtual Page Number Bits: ____ 29 ____

2. How many **virtual** pages exist in the system? [4]

____ 2^{29} pages ____

3. How many **physical** pages exist in the system? [4]

Memory = 128 GB, page size = 8 KB so pages = $128 \text{ GB} / 8 \text{ KB} = 2^{37} / 2^{13} = 2^{24}$

____ 2^{24} pages ____

4. If the system has a single-level page table with 8B entries, how large must the page table be to map all the system's virtual memory? [4]

The number of entries in the page table = number of virtual pages = 2^{29}

Each entry is 8B large. So $8\text{B} * 2^{29}$ should equal the total size = 2^{32} bytes or 4 GB for the page table.

5. If we switch to a hierarchical page table such that each page table level must fit in a single page, how many page table levels would we need to map all of the system's virtual memory? Assume page table entries are still 8B each. [4]

To map the total virtual memory 2^{29} pages. For pages the page size is 8KB and entries are 8B, so entries per page is $8\text{KB} / 8\text{B} = 2^{13} / 2^3 = 2^{10}$. For a hierarchical table we need $\log_2(2^{29}) / \log_2(2^{10}) = 2.9$ so about 3 levels.

Problem 3: Miss Me With That [20 points] (L20)

Consider a byte-addressable system which includes a cache with the following specifications:

- 2-Way Set-Associativity
- Block Size: 8 bytes
- Cache Size: 32 bytes
- LRU replacement policy

Given the following sequence of cache accesses, determine whether each access is a hit or a miss and then classify each of the misses as one of compulsory, capacity, or conflict.

Additionally, break down each address to show the tag and set index in order to help see what is in the cache. The last three columns are optional.

Set 0 [8,0] set 1 [9,11]

Address	Tag (Binary)	Set Index	Hit / Miss?	Type (N/A if Hit)	Infinite Cache	Fully Associative	Set Associative
0x3B 59/8=7	0011 1 011	1	Miss	Compulsory	x	x	x
0x40 64/8=8	0100 0 000	0	Miss	Compulsory	x	x	x
0x03 3 /8=0	0000 0 011	0	Miss	Compulsory	x	x	x
0x5B 91/8=11	0101 1 011	1	Miss	Compulsory	x	x	x
0x00 0/ 8= 0	0000 0 000	0	Hit	N/A	x	x	x
0x2F 47/8=5	0010 1 111	1	Miss	Compulsory	x	x	x
0x3E 62/8=7	0011 1 110	1	Miss	Capacity	x	x	x
0x59 89/8=11	0101 1 001	1	Miss	Conflict	x	x	x
0x69 105/8=13	0110 1 001	1	Miss	Compulsory	x	x	x
0x4E 78/8=9	0100 1 110	1	Miss	Compulsory	x	x	x

0x5D 93/8=11	0101 1 101	1	Miss	Conflict	x	x	x
0x46 70/8=8	0100 0 110	0	Hit	N/A	x	x	x
0x24 36/8=4	0010 0 100	0	Miss	Compulsor y	x	x	x
0x07 7/ 8=0	0000 0 111	0	Miss	Conflict	x	x	x

Problem 4: Two-level page table [20 points] (L22)

You are checking the correctness of a system with virtual memory and no caches. The system uses a 2-level page table scheme with the following partitioning for 12-bit virtual addresses:

L1 Page Table Index	L2 Page Table Index	Page Offset
Bits 11-8	Bits 7-4	Bits 3 - 0

The page table translates virtual addresses into physical addresses of the following form:

Physical Page Number	Page Offset
Bits 10 - 4	Bits 3 - 0

Each page table entry is **1 byte** wide, and it is stored in (physical) memory. The format of each page table entry is as follows:

Valid bit	Page Number
Bit 7	Bits 6 - 0

(Note: this implies that level-2 page tables must start at a page boundary, since we are only storing a page number in level-1 entries, rather than a full address). A snippet of the machine's physical memory is given in the next page (contents are laid out 16 bytes per row, the first entry being the lowest-order byte. For example, address 0x1 contains the byte 0xf2). You also know that the page table **base register** points to physical address **0x110**.

Translate the two virtual addresses in the next page into physical addresses. Specify the address (not the index!) of the Level 1 (L1) page table entry of interest and its value, and do the same for second-level page tables. Write “–” to indicate an invalid or unknown entry/address. [Hint: You need to use page table entry size and page table index to determine the location of a page table entry in memory.]

Virtual Address	Address of L1 Page Table Entry	Content of L1 Page Table Entry	Address of L2 Page Table Entry	Content of L2 Page Table Entry	Physical Address	Content of Physical Address
0x32f L1 addr: 0x110+3 L1 entry: @0x113 =0x86 L2 addr: 0x110+2	0x113	0x86	0x112	0x62	0x2f	99
0x1c6	0x111	0x00	-	-	-	-

Snippet of physical memory (All values in hexadecimal):

Address	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+a	+b	+c	+d	+e	+f
0x0	f0	f2	00	00	00	00	00	00	00	a4	40	23	44	12	12	12
0x10	86	8c	c9	4f	00	3f	42	00	00	00	8a	a3	9f	91	00	19
0x20	00	af	00	14	00	12	00	00	5f	00	40	00	00	90	33	99
0x30	c6	e8	39	a5	3a	20	b0	ff	72	79	e7	4e	be	41	70	c4
0x40	6a	91	f2	40	94	62	19	9c	97	04	9f	95	cd	30	1c	9e
0x50	00	86	19	4a	50	b0	3f	16	e8	f9	4e	06	1a	75	da	e2
0x60	80	99	82	00	8f	90	91	92	c4	d3	00	00	34	00	00	00
0x70	d2	ff	78	ae	e8	cb	57	56	f8	b3	55	08	d3	bd	9c	cd
0x80	46	30	c9	4f	44	3f	42	70	22	62	8a	b9	78	28	40	19
0x90	61	07	11	3	00	00	00	ca	00	00	00	00	00	80	09	00
0xa0	00	b8	00	ba	00	b9	00	00	00	40	00	0b	c0	00	aa	00
0xb0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0xc0	00	00	c3	b4	b5	00	b6	00	00	00	00	00	00	00	00	00
0xd0	54	94	09	68	0d	fd	db	46	6e	85	dd	a3	01	f7	f7	13
0xe0	ad	ac	10	a1	ed	83	92	42	99	40	c5	ca	df	68	20	8f
0xf0	12	d1	b3	44	00	55	00	00	00	b4	9c	ae	a0	97	b0	c2
0x100	1f	00	02	00	e3	00	00	54	00	00	20	34	76	00	50	30
0x110	90	00	00	86	00	00	a3	a2	a1	c0	00	00	00	00	00	00
0x120	6f	ef	01	aa	70	22	b7	30	ce	67	05	97	b0	b7	5d	6d
0x130	00	34	00	00	55	55	55	67	00	88	24	00	4a	01	00	3c

Problem 5: Course Surveys [5 points]

1. Take a screenshot that shows you did at least one end-of-term course evaluation for EECS 370. We really do read all of these and that can be very useful in helping with future terms. **[5]**

Your responses have been submitted successfully!

Shikha Nukala, thank you for completing the evaluation for EECS 370-001: Intro Computer Org.

You will receive an email confirmation that you

EECS 370-026: Intro Computer Org
● Open Ends on: 2024-12-10

EECS 388-002: Intro Comp Security
● Open Ends on: 2024-12-10

EECS 388-017: Intro Comp Security
● Open Ends on: 2024-12-10

View All

Start now

Start now

Start now

Problem 6: I'm having cache flow problems [20 points, Group] (L19)

Consider the following access pattern: A, B, C, D, A. Assume that A, B, C and D are memory addresses each of which are in a different block of memory. Further, assume A, B, C and D are generated in a uniformly random way (each block is equally likely) and that a true LRU replacement algorithm is used. *To receive credit you must briefly show your work.*

What is the probability that the second instance of "A" will be a hit if:

1. The cache has 2 lines and is fully-associative [3].

The cache is fully-associative so blocks can be placed in any place. The cache has two lines so only two blocks can be in the cache at a given time. If we access A then B the cache looks like {A, B} loaded from memory. Then accessing C we find the LRU which is A so we move A from cache to nowhere and load C so we have {C,B}. Then we get D so it becomes {C,D}. Since A can never be present in the cache at this time with the access pattern it is always a miss for the second instance so the probability of a hit is 0%.

2. The cache has 4 lines and is fully-associative [3].

For the first four accesses we get a cache that looks like {A,B,C,D} and then when we get a second instance of accessing A we see that block of memory A is always in the cache at this access pattern point so the probability of a hit for this instance is 100%.

3. The cache has 2 lines and is direct-mapped [3].

The cache is directly-mapped so there is a way to differentiate where to put a block in cache based on where it resides in memory. Either way we see that since the cache has only 2 lines and the memory blocks C and D are accessed before A again, there is a very low chance that the mapping for B, C, and D are the same while A is different for memory. If this was the case then we know A is mapped first. Then if B is mapped to the other line it's a $\frac{1}{2}$ chance. If C is mapped to B we also have a $\frac{1}{2}$ chance. If D is mapped to C then it is also a $\frac{1}{2}$ chance. So the probability of A still being present in the cache is $\frac{1}{2} * \frac{1}{2} * \frac{1}{2} = \frac{1}{8}$.

4. The cache has 4 lines and is direct-mapped, *you may leave your answer in a fraction.* [5]

Since there are 4 lines and we have direct-mapping but we do not know how the direct-mapping is done, we can use probability to find the chance of the second A instance being a hit. For the first round, A is loaded into the cache at it's correct spot. Next, B is loaded. The probability that B is not mapped to the spot of A is $\frac{3}{4}$. Then C is mapped next. The probability that C is not mapped to the spot of A is also $\frac{3}{4}$. Next D is mapped and the probability of not mapping to A is also $\frac{3}{4}$. So the probability of A still being in the cache at the time of the second iteration is $\frac{3}{4} * \frac{3}{4} * \frac{3}{4} = \frac{27}{64}$

Now, assume the access pattern is this instead: "A, B, C, C, A". Again, Assume that A, B, and C are memory addresses each of which are in a different block of memory. Further, assume A, B and C are generated in a uniformly random way (each block is equally likely) and that a true LRU replacement algorithm is used.

What is the probability that the second instance of "A" will be a hit if:

5. The cache has 2 lines and is direct-mapped **[3]**.

The cache has two lines and is direct-mapping. We do not know the method of direct-mapping so we will use the probability of each round to determine the probability. First A is loaded in the cache. Then B has a $\frac{1}{2}$ chance of not mapping to A's position. Then C has a $\frac{1}{2}$ chance of not mapping to A's position. The next round C is a hit since it's already present so the probability of A still being present in the cache and generating a hit for the second instance is $\frac{1}{2} * \frac{1}{2} = \frac{1}{4}$.

6. The cache has 4 lines and is two-way set associative? **[3]**

Here we have two sets with two lines each. If a block of memory is mapped to one set then that memory can go to any line in that set. Therefore we can use probability to find the status of block A. First A is loaded into the cache in set x. Then to load B it doesn't matter where B maps to because it will not overwrite A's position. For C the only scenario where C overwrites A is if C maps to set x and B mapped to set x. This scenario has $\frac{1}{2} * \frac{1}{2} = \frac{1}{4}$ chance of happening. Then the second instance of A is encountered. So A is a hit in $\frac{3}{4}$ of possible scenarios.